CS3210: Operating Systems

Lecture 12 — Schedulers

John Kim
Georgia Institute of Technology

February 24, 2026

Contents

1 Motivation: Why Do We Need Schedulers? 3
1.1 The Core Problem: Multiplexing . . . 3
1.2 What Does a Scheduler Do? . . . 3
1.3 What Can Be Scheduled? . . . 3
1.4 The Goal of a Scheduler . . . 3

2 Scheduling Goals 3
2.1 Throughput Scheduling . . . 4
2.2 Response Time Scheduling . . . 4
2.3 Latency SLO Attainment . . . 4
2.4 Cost Minimization . . . 4
2.5 The Systems Conjecture . . . 4

3 Latency Distribution Functions 4
3.1 Probability Distribution Function (PDF) . . . 4
3.2 Cumulative Distribution Function (CDF) . . . 5
3.3 Tail Latency . . . 5

4 Scheduler Design Space 5
4.1 Scheduler Scenarios . . . 5
4.2 Design Considerations: Costs and Benefits . . . 6
4.3 The Full Scheduling Design Space . . . 6
4.4 Scheduling Is Difficult in Practice . . . 7

5 Scheduling Algorithms 7
5.1 Round Robin (RR) . . . 7
5.2 Linux Completely Fair Scheduler (CFS) . . . 7
5.3 Scheduling Policies in Linux . . . 8
5.4 Real-Time Scheduling . . . 8
5.5 Characterizing Processes . . . 9

6 Context Switching in xv6 9

6.1 The xv6 Scheduler . . . 9
6.2 Context Switching: Big Picture . . . 10
6.3 Context Switching: Implementation Challenges . . . 10
6.4 Two Kinds of Context Switch . . . 11
6.5 xv6 Process Kernel Context . . . 11
6.6 The `swtch` Function . . . 11
6.7 yield . . . 12
6.8 Giving Up the CPU: Protocol . . . 12
6.9 sched and scheduler: Coroutines . . . 12

7 Summary **12**

8 References **13**

1 Motivation: Why Do We Need Schedulers?

1.1 The Core Problem: Multiplexing

A scheduler is needed any time there are **multiple resource consumers** competing for a **scarce set of shared resources**. This is the fundamental problem of *multiplexing*.

Consider:

- With 1 process and 1 CPU — no scheduling decisions needed.
- With 2 processes and 2 CPUs — still no contention, trivially satisfied.
- With more processes than CPUs — a scheduler is required to decide *who runs when*.

Scheduling is deeply tied to **economic theory**: allocating scarce resources among competing consumers to maximize some measure of value.

1.2 What Does a Scheduler Do?

The scheduler is one of the key OS mechanisms enabling:

- **Process isolation** — ensuring processes don't interfere with each other.
- **Resource multiplexing** — sharing CPU time among processes, which implies:
  - *Fairness:* each process gets a reasonable share.
  - *Resource utilization:* the CPU is not wasted sitting idle.

1.3 What Can Be Scheduled?

The scheduling abstraction is broadly applicable. In different contexts, schedulers manage:

- **Processes/threads** on CPUs (the classic OS case)
- **Disk I/O requests**
- **Network packets/connections**
- **Cloud system resources** (VMs, containers)
- **AI workloads** (GPU time for training vs. inference)

1.4 The Goal of a Scheduler

High-level goal: Maximize the amount of work we care about.

This sounds simple but is deceptively complex — "work we care about" depends entirely on the workload and policy. This leads to a rich design space of scheduling algorithms.

2 Scheduling Goals

Different systems optimize for very different objectives. There is no universal best scheduler.

2.1 Throughput Scheduling

Use case: Disk I/O scheduling.

- Performance varies by *orders of magnitude* depending on access patterns (sequential vs. random I/O).
- **Elevator scheduling** (SCAN): The disk head sweeps back and forth, servicing requests in order of disk position. Reduces total seek distance.
  - *Implicit assumption:* There is an *actuation delay* when changing direction.
  - **Declarative scheduling** (e.g., Coda elevators): Instead of specifying *how* to schedule, specify *what outcome* is desired. The system figures out the optimal execution.

2.2 Response Time Scheduling

Use case: Social media feeds, web servers, interactive applications.

Minimize the time between a user action and the system's response. Even if throughput is lower, a snappy experience is preferred.

2.3 Latency SLO Attainment

Use case: Ad serving, LLM inter-token generation latency, AR/VR.

Maximize the fraction of requests that complete within a predefined **Service Level Objective (SLO)** deadline. This is framed as an **optimization problem**.

2.4 Cost Minimization

Use case: Cloud computing.

Minimize resource cost (e.g., dollars spent on CPU hours) while still meeting performance requirements.

2.5 The Systems Conjecture

A key insight in systems design: **you cannot simultaneously optimize for all three of:**

Simplicity	**Performance**	**Cost**
Easy to implement	High throughput / low latency	Cheap to run

Any real system must pick (at most) two. A simple algorithm is rarely the fastest; a high-performance algorithm is rarely cheap; cheap systems are rarely simple or fast.

3 Latency Distribution Functions

To reason about latency SLOs, we need tools to characterize latency distributions.

3.1 Probability Distribution Function (PDF)

$y = P(X = x)$: The probability that a request's latency is *exactly* x ms.

3.2 Cumulative Distribution Function (CDF)

$y = P(X \leq x)$: The probability that a request's latency is *at most* x ms.

A CDF plot with latency on the x-axis and $P(X \leq x)$ on the y-axis shows what fraction of requests complete within each latency value.

3.3 Tail Latency

Tail latency refers to high-percentile latencies — e.g., the 99th percentile (p99) latency is the value x such that 99% of requests complete within x ms.

Example: Suppose a CDF shows:

- Median (p50) latency: 16.19 ms
- p90 latency: 29.40 ms
- p99 latency: 35.67 ms
- Latency SLO: 36 ms

The scheduler's job (in SLO scheduling) is to ensure that the p99 (or whatever SLO target) stays within the deadline. Tail latency is what *users notice* and what SLAs are written against.

4 Scheduler Design Space

4.1 Scheduler Scenarios

Two contrasting workloads:

- **Long-running jobs**: A simple FCFS (First Come, First Served) scheduler may be fine. Jobs run to completion without much switching overhead.
- **Very short jobs**: A **preemptive** scheduler is better. Jobs are short enough that frequent switching is worthwhile to improve responsiveness.

Why does a different scheduler excel for each? Because the cost/benefit of preemption differs: for long jobs, context switching overhead is negligible relative to job length; for short jobs, responsiveness matters more than throughput.

Advanced Scheduling Strategies

- **Delayed scheduling** (EuroSys'10): Intentionally delay scheduling to get better locality or packing. This is a *non-work-preserving* strategy (CPU may be idle briefly while waiting for a better assignment).
- **Power of two choices**: Randomly sample two queues and schedule on the less loaded one. Simple heuristic with strong theoretical guarantees.
- **Spatio-temporal bin packing** (TetriSched, EuroSys'16): Pack jobs into resources in both space (which machines) and time (when) to maximize utilization, like fitting Tetris pieces.

Key concept: *Fluidity* — how fluidly can jobs be split across resources? Think of it like sand (very fine-grained, flows anywhere) vs. water (flows but has surface tension) vs. blocks (coarse,

rigid). This is the idea of *spatio-temporal granularity*: how small a unit of work can be moved, and over what time horizon.

4.2 Design Considerations: Costs and Benefits

When building a scheduler, weigh:

Costs of scheduling:

- **Context switching overhead:**
  - *Switching computation:* CPU time spent saving and restoring register state.
  - *Caching costs:* After a context switch, the new process's data is likely not in cache (TLB and cache cold start). This can dominate for short time quanta.
- **Scheduler time complexity:** A complex scheduler (e.g., one that computes optimal assignments) takes more CPU time itself.
- **Predictability:** Real-time operating systems (RTOS) need deterministic scheduling latency. Complex algorithms introduce jitter.

Benefits of scheduling:

- Better resource utilization (CPUs don't sit idle while runnable processes wait).
- More CPU time directed to high-priority/high-value tasks.
- Stronger Quality of Service (QoS) guarantees.

4.3 The Full Scheduling Design Space

Designers must choose along many axes:

- **Throughput vs. latency:** Optimize for total work done, or for individual request response time?
- **Fairness:** Time quota (equal time slices), epoch-based fairness, weighted fairness in heterogeneous contexts.
- **QoS / Priority:** Should some processes be more important? (Linux `nice` values, real-time priorities.)
- **Preemptive vs. Cooperative:**
  - *Preemptive:* The OS can forcibly remove a process from the CPU (via timer interrupt).
  - *Cooperative:* Processes voluntarily yield the CPU. Simpler, but a misbehaving process can starve everyone else.
- **Global vs. Local:** Does the scheduler have a system-wide view (global), or does each CPU decide independently (local)? Global is more optimal but harder to scale.
- **Scalability:** Can the scheduler handle 1M processes? $O(n)$ algorithms that scan a full process table don't scale.
- **Granularity (time quantum):** 10ms? 100ms? Dynamic? Shorter quanta improve responsiveness but increase overhead.

- **Constraints:** Real-time deadlines (e.g., flight control systems must meet hard deadlines).
- **Other concerns:** Resource starvation, performance consolidation in cloud environments.

4.4 Scheduling Is Difficult in Practice

No perfect, universal scheduling policy exists. Goals are contradictory:

Goal A	vs. Goal B
Maximize throughput	Minimize latency
Minimize response time	Maximize scalability
Maximize fairness	Maximize scalability
Maximize resource utilization	Minimize power consumption

This motivates many different algorithms: FCFS, Shortest Job First (SJF), Priority Scheduling, Round-Robin (RR), Multi-level Queue, Multi-level Feedback Queue (MLFQ), MinJCT, and more.

5 Scheduling Algorithms

5.1 Round Robin (RR)

Round-Robin is the algorithm used in xv6. Each process is assigned a fixed time unit (*quantum* or *time slice*), and processes are served in circular order.

Advantages:

- **Simple:** Fixed time unit per process, no priority computations.
- **Starvation-free:** Since there is no priority, every process is eventually served.

Disadvantages:

- Does not distinguish urgency or priority between processes.
- **Deadline-unaware:** Cannot meet real-time deadlines.
- Does not handle the disparity between throughput-oriented and latency-sensitive workloads.

Example (Quantum = 3): With processes P1(0,1), P2(0,2), ..., P10(11,1), each process gets 3 time units before being preempted. Short processes finish quickly; long processes have high turnaround time but no starvation.

5.2 Linux Completely Fair Scheduler (CFS)

Motivating Observation

Processes that *constantly consume the full CPU* are likely lower priority than those that mostly sleep and wake up for short bursts.

Example: A network server (e.g., nginx) needs low latency to handle incoming requests, but spends most of its time sleeping waiting for connections. A deep learning training job, by contrast, runs continuously. The network server should be rewarded for sleeping and given high priority when it wakes.

Key Idea: *Reduce the priority of processes that always consume full CPU.* Conversely, boost priority of processes that frequently sleep. We're incentivizing processes to not constantly need the CPU.

Weighted Fair Queuing (WFQ)

CFS is Linux's implementation of **Weighted Fair Queuing (WFQ)**:

- Resources (temporal, spatial) are shared *proportionally* to each process's **weight**.
- Example: If process A has weight 2 and process B has weight 1, then A gets 2/3 of the CPU and B gets 1/3.
- Weights in Linux correspond to `nice` values (range -20 to $+19$; lower = higher priority).

CFS and Sleeping Processes

What happens when a process sleeps and misses its CPU time? Naively, when it wakes up, it has a massive backlog of "owed" CPU time and could monopolize the CPU.

CFS handles this with **virtual runtime** (`vruntime`): each process tracks how much CPU time it has consumed (normalized by weight). CFS always runs the process with the *smallest* `vruntime`. When a process wakes from sleep, its `vruntime` is set to the *current minimum*, so it gets priority without a runaway effect.

Graphical intuition (from lecture): The priority of a sleeping inference server oscillates up and down (it gets high priority when it wakes, then drops as it consumes CPU). The DL training job continuously decays in priority since it always consumes full CPU.

5.3 Scheduling Policies in Linux

Linux supports multiple scheduling policies, selectable per-process:

Policy	Description
`SCHED_OTHER`	Normal time-sharing (default); uses CFS.
`SCHED_IDLE`	Very low priority background tasks.
`SCHED_BATCH`	CPU-intensive batch processes; longer time slices.
`SCHED_FIFO`	Real-time, first-in first-out; runs until it blocks.
`SCHED_RR`	Real-time round-robin with fixed time quantum.

`SCHED_FIFO` and `SCHED_RR` are real-time policies and preempt all `SCHED_OTHER` processes.

5.4 Real-Time Scheduling

Real-time systems are divided into two categories:

- **Hard real-time:** Critical tasks *must* complete within a guaranteed time bound. Missing a deadline is a system failure (e.g., airbag deployment, flight control).
  - *Note:* Hard real-time is *impractical* with secondary storage (disk) or virtual memory. Why? Because page faults and disk I/O have unpredictable, potentially unbounded latency that can make deadlines impossible to guarantee.

- **Soft real-time:** Critical processes receive priority over less critical ones, but occasional deadline misses are tolerable (e.g., video streaming, interactive audio). The optimization function setup is different here.

Soft real-time requires two things:

1. **Priority scheduling** (urgency-aware): The system must know which processes are time-critical and prefer them.
2. **Low dispatch latency:** After a high-priority process becomes runnable, it must actually run quickly. This is hard to guarantee because:
   - System calls may not be preemptible (e.g., the kernel cannot safely interrupt a system call mid-execution without complex locking).
   - Linux achieves low dispatch latency via the "fully preemptible kernel" configuration.

5.5 Characterizing Processes

Not all processes are the same. Key distinctions:

- **CPU-bound:** Spends most of its time computing (e.g., scientific simulation, video encoding).
- **I/O-bound:** Spends most of its time waiting for I/O (e.g., database, file server).
- **Interactive processes:** Respond to user input (e.g., vim, emacs). Require low latency.
- **Batch processes:** Run without user interaction on a schedule (e.g., cron jobs). Throughput matters more than latency.
- **Real-time processes:** Have strict timing requirements (e.g., audio playback, video streaming, AR/VR, chatbot).
  - Present an interesting *throughput vs. latency tradeoff*: the system cares about both queries-per-second (qps) and per-request real-time success metrics like Token Budget Time (TBT) for LLM inference.

6 Context Switching in xv6

6.1 The xv6 Scheduler

xv6 uses a simple **round-robin** scheduler. Its main loop:

Listing 1: xv6 scheduler (proc.c)

```
// Loop over process table looking for process to run.
acquire(&ptable.lock);
for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
    if(p->state != RUNNABLE)
        continue;

    // Switch to chosen process. It is the process's job
    // to release ptable.lock and then reacquire it
    // before jumping back to us.
    proc = p;
    switchuvm(p);
```

```
    p->state = RUNNING;
    swtch(&cpu->scheduler, p->context);
    switchkvm();

    // Process is done running for now.
    // It should have changed its p->state before coming back.
    proc = 0;
}
release(&ptable.lock);
```

Steps:

1. Scan the process table for a `RUNNABLE` process.
2. Set the current per-CPU process pointer (`proc = p`).
3. Call `switchuvm(p)` to switch to the process's page table.
4. Mark the process as `RUNNING`.
5. Call `swtch` to perform the kernel context switch.
6. After the process yields, call `switchkvm()` to switch back to kernel page table.
7. Reset `proc = 0` and continue scanning.

6.2 Context Switching: Big Picture

What we want to do: Switch directly from process A (e.g., `shell`) to process B (e.g., `cat`) by swapping their kernel contexts.

What we actually have to do: There is no direct user-to-user context switch. The full path is always:

$$\text{shell (user)} \xrightarrow{\text{save user ctx}} \text{shell (kernel)} \xrightarrow{\text{swtch}} \text{scheduler} \xrightarrow{\text{swtch}} \text{cat (kernel)} \xrightarrow{\text{restore user ctx}} \text{cat (user)}$$

This means **two** calls to `swtch`:

1. Process's kernel context → CPU scheduler context (via `sched/yield`).
2. CPU scheduler context → next process's kernel context (via `scheduler`).

Key rule: xv6 *never* directly switches from user-space to user-space.

6.3 Context Switching: Implementation Challenges

- **How to switch from one process to another?** → Context switching (save/restore registers + stack).
- **How to make context switching transparent?** → Timer interrupts force preemption without process cooperation.
- **How to switch among processes running concurrently?** → Locking (specifically `ptable.lock`) prevents races on the process table.

- **How to coordinate processes?** → Sleeping on events (e.g., a process sleeps waiting for a pipe to have data, or for a child to exit).

6.4 Two Kinds of Context Switch

1. **Process kernel context → CPU scheduler context:** When the running process gives up the CPU (via `yield`, `sleep`, or `exit`).
2. **CPU scheduler context → process kernel context:** When the scheduler picks the next process to run.

6.5 xv6 Process Kernel Context

- Every xv6 process has its own **kernel context**: a kernel stack and a register set (including `%esp` and `%eip`).
- Every CPU has its own **scheduler thread** (with its own kernel stack).
- Switching threads = saving current registers + loading next registers, including `%esp` (stack pointer) and `%eip` (instruction pointer).

6.6 The `swtch` Function

```
void swtch(struct context**, struct context*);
```

`swtch` is a low-level assembly routine that:

- **Does not know about threads** — it purely saves and loads register sets called *contexts*.
- When a kernel thread gives up the CPU, it calls `swtch` to save itself and return to the scheduler context.
- A `struct context*` is stored on the kernel stack.
- Pushes current registers onto the old kernel stack; saves `%esp` to `*old`.
- Copies new context pointer into `%esp`; pops previous registers; `ret` resumes at the saved `%eip`.

Detailed steps of `swtch`:

1. Load arguments off the stack into `%eax` (old) and `%edx` (new) *before* modifying `%esp`.
2. Save only **callee-saved registers**: `%ebp`, `%ebx`, `%esi`, `%edi`. Push them; save `%esp` as `*old`.
3. Note: `%eip` was already saved by the `call` instruction (return address), positioned just before `%ebp` on the stack.
4. Switch stacks: `movl %edx, %esp`.
5. Restore new context: pop `%edi`, `%esi`, `%ebx`, `%ebp`; then `ret` (which pops the saved `%eip`).

Why only callee-saved registers? Because `swtch` is a *voluntary* yield — the kernel thread calls `swtch` explicitly as a function call. By the C calling convention, caller-saved registers are the caller's responsibility; they were saved by the code before calling `swtch`. Only callee-saved registers need to be preserved across the call boundary.

6.7 yield

`yield` is called at the end of each timer interrupt to preempt the current process.

The call chain is:

$$\texttt{yield} \rightarrow \texttt{sched} \rightarrow \texttt{swtch}$$

`yield` switches from `proc->context` to `cpu->scheduler`.

6.8 Giving Up the CPU: Protocol

Any process giving up the CPU (via `yield`, `sleep`, or `exit`) must follow these steps:

1. **Acquire `ptable.lock`:** Protects process state.
2. **Release any other locks:** To avoid holding locks across a context switch, which could cause deadlocks.
3. **Update its own state:** Set to `RUNNABLE` (yield) or `SLEEPING` (sleep) or `ZOMBIE` (exit).
4. **Call `sched`:** Which calls `swtch` to hand off to the scheduler.

`yield`, `sleep`, and `exit` all follow this same pattern.

6.9 sched and scheduler: Coroutines

- A kernel thread always gives up in `sched` and switches to the *same location* in the scheduler (wherever the scheduler last called `swtch`).
- The scheduler almost always switches to a process in `sched`.
- This back-and-forth forms a pattern of **coroutines**: two execution contexts that cooperatively pass control back and forth at well-defined handoff points.
- **Exception:** `forkret` — when a newly created process runs for the first time, there is no saved `sched` context to return to. Instead, the process starts at `forkret`, which performs initialization and then returns to user space.

7 Summary

- **Schedulers** are needed for resource multiplexing when demand exceeds supply.
- Scheduling goals vary: throughput, latency, SLO attainment, fairness, cost — and these goals conflict.
- The **Systems Conjecture**: you cannot have simplicity, performance, and low cost simultaneously.
- **CFS** (Linux) implements Weighted Fair Queuing, dynamically adjusting priorities based on CPU consumption.
- **Real-time scheduling** comes in hard (guaranteed deadlines) and soft (best-effort priority) flavors.
- Linux exposes multiple scheduling policies (`SCHED_OTHER`, `SCHED_FIFO`, `SCHED_RR`, etc.) to support different workload types.

- In **xv6**: context switching always goes through the kernel scheduler (never user → user directly). `swtch` saves and restores callee-saved registers. `sched`/`scheduler` operate as coroutines.

8 References

- Arpaci-Dusseau, R. H.; Arpaci-Dusseau, A. C. (2014). "Multi-level Feedback Queue." *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books.
- Silberschatz, A.; Galvin, P. B.; Gagne, G. (1999). *Applied Operating System Concepts*. John Wiley & Sons.
- Lozi, J.-P. et al. "The Linux Scheduler: a Decade of Wasted Cores." EuroSys 2016.
- IBM DeveloperWorks. "The Completely Fair Scheduler." `https://www.ibm.com/developerworks/linux/library/l-completely-fair-scheduler/`

Personal Notes